

Skill-13

Name: Simran Jangid

Rollno: 2520030166

Section: 8

Experiment No. 13 (Q13)

Title: Input Redirection and Output Redirection Handling in a Custom Shell

Aim:

To parse input redirection, open files, redirect stdin, handle missing files, restore streams, and test file inputs. To parse output redirection, create output files, redirect stdout, handle permissions, verify output data, and test redirection.

Objective:

Part 1 – Input Redirection

- To parse the input redirection operator '<' from a command line.
- To open the specified file for reading.
- To redirect standard input so the command reads from the file.
- To handle missing or unreadable input files gracefully.
- To restore the original standard streams after execution.
- To test commands that take their input from a file.

Part 2 – Output Redirection

- To parse the output redirection operators '>' and '>>' from a command line.
- To create or open the output file with the correct flags.
- To redirect standard output so the command writes into the file.
- To handle permission errors when the file cannot be written.
- To verify the data actually written to the output file.
- To test truncating, appending and invalid redirection cases.

Theory:

1. Standard Streams and File Descriptors

Every process in UNIX starts with three open file descriptors: standard input (0), standard output (1) and standard error (2). By default these are connected to the terminal. Redirection means replacing the file a descriptor points to, without the command itself knowing about the change.

2. Input Redirection

Input redirection uses the '<' operator, as in "wc -l < input.txt". The shell opens the named file for reading and attaches it to descriptor 0, so the command reads the file contents instead of the keyboard. The command itself is unchanged; only its input source is replaced.

3. Parsing the Redirection Operators

The input line is tokenized on spaces and tabs. A token equal to '<', '>' or '>>' is not part of the argument vector; instead the token that follows it is stored as the redirection target. Everything else is appended to the argument array passed to `execvp()`. If an operator appears with no file name after it, a syntax error is reported.

```
if (strcmp(token, "<") == 0) { token = strtok(NULL, " \t"); cmd->inputFile = token; }
else if (strcmp(token, ">") == 0) { token = strtok(NULL, " \t"); cmd->outputFile = token; }
```

4. Opening Files with open()

The `open()` system call returns the lowest available file descriptor for a file. For input, the file is opened with `O_RDONLY`. For output, `O_WRONLY | O_CREAT` is used, combined with `O_TRUNC` to overwrite the file or `O_APPEND` to add to it, and the mode `0644` gives the created file read/write permission for the owner and read permission for others.

```
int fd = open(fileName, O_RDONLY);
int fd = open(fileName, O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

5. Redirecting Streams with dup() and dup2()

`dup2(fd, STDIN_FILENO)` closes descriptor 0 if it is open and makes it a copy of `fd`, so all reads from standard input now come from the file. `dup(STDIN_FILENO)` is called first to keep a spare copy of the original stream, which is what makes restoration possible later.

```
savedStdin = dup(STDIN_FILENO);    /* keep a copy of the terminal */
dup2(fd, STDIN_FILENO);           /* stdin now reads from the file */
```

6. Handling Missing Files and Permission Errors

If `open()` fails it returns `-1` and sets `errno`. A missing input file gives `ENOENT` ("No such file or directory") and a write attempt on a read-only file gives `EACCES` ("Permission denied"). In both cases the shell reports the error using `strerror(errno)` and skips execution instead of terminating, so the user can continue entering commands.

7. Restoring the Standard Streams

After a redirected command finishes, the saved descriptors are copied back with `dup2()` and then closed. Without this step, every later command in the same process would keep reading from or writing to the redirected file. Restoring also prevents file descriptor leaks, since each `dup()` consumes an entry in the process descriptor table.

```
dup2(savedStdin, STDIN_FILENO); close(savedStdin);
dup2(savedStdout, STDOUT_FILENO); close(savedStdout);
```

8. Redirection in the Child Process

The redirection is applied in the child created by `fork()`, after the fork but before `execvp()`. Because the child has its own copy of the descriptor table, the parent shell's own terminal input and output remain untouched no matter what the command does. The file descriptors survive `execvp()`, which is precisely why the replaced program inherits the redirection.

9. Verifying the Redirected Output

Because a redirected command prints nothing to the terminal, the result is confirmed by reopening the output file and displaying its contents. This also distinguishes truncation from appending: `>` replaces the previous contents, while `>>` leaves them and adds new data at the end.

Algorithm:

Part 1 – Input Redirection

- Start the program.

- Read the command line entered by the user.
- Tokenize the line on spaces and tabs.
- If a token is '<', store the following token as the input file name.
- Store all remaining tokens in the argument array.
- Fork a child process.
- In the child, open the input file with O_RDONLY.
- If open() fails, report the error and skip execution.
- Otherwise save standard input using dup() and redirect it using dup2().
- Execute the command with execvp() so it reads from the file.
- Wait for the child and restore the saved streams.
- Test with valid files and with a missing file.
- Stop the program.

Part 2 – Output Redirection

- Start the program.
- Read the command line entered by the user.
- If a token is '>' or '>>', store the following token as the output file name and set the append flag accordingly.
- Store all remaining tokens in the argument array.
- Fork a child process.
- In the child, open the output file with O_WRONLY | O_CREAT and either O_TRUNC or O_APPEND, using mode 0644.
- If open() fails because of permissions, report the error and skip execution.
- Otherwise save standard output using dup() and redirect it using dup2().
- Execute the command with execvp() so its output is written into the file.
- Wait for the child and restore the saved streams.
- Reopen the output file and display its contents to verify the written data.
- Test truncation, appending and a permission-denied case.
- Stop the program.

Functions Used:

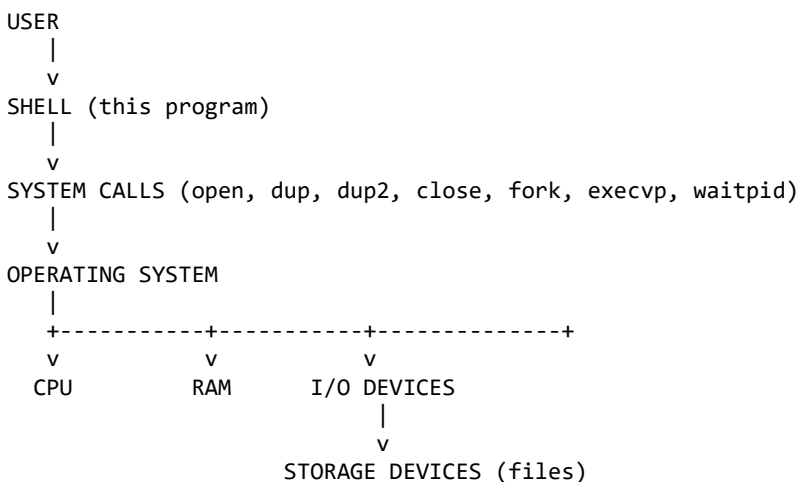
Function	Purpose
main()	Starting point of the program
parseRedirection()	Scans the input line for '<', '>' and '>>' and separates the redirection targets from the command arguments
printRedirectionLayout()	Displays the parsed command along with its input and output targets
redirectInput()	Opens the input file and redirects standard input to it
redirectOutput()	Creates/opens the output file and redirects standard output to it
restoreStreams()	Restores the original standard input and output from the saved descriptors
executeCommand()	Forks a child, applies the redirections and executes the command
verifyFileContents()	Reads back the output file to verify the redirected data

Function	Purpose
createSampleInputFile()	Creates a small data file used by the input redirection tests
testInputRedirection()	Tests valid input files and a deliberately missing file
testOutputRedirection()	Tests file creation, append mode and a permission-denied case
open()	Opens or creates a file and returns a file descriptor
dup()	Duplicates a file descriptor so the original stream can be saved
dup2()	Redirects standard input/output to the opened file descriptor
close()	Closes a file descriptor once it is no longer needed
fork()	Creates a child process in which the redirection is applied
execvp()	Replaces the child process image with the requested command
waitpid()	Waits for the child process to finish before continuing
chmod()	Changes file permissions to create the permission-denied test case
strerror()	Converts the errno value into a readable error message

Hardware and Operating System Relationship

Hardware consists of physical components such as the CPU, RAM, storage devices, keyboard, mouse and display. The Operating System acts as an interface between application software and hardware. It manages processes, memory, files, devices and system resources.

Relationship



How OS Interacts with Hardware

- **Process Management** – The OS creates a child process for each redirected command and schedules it.
- **Memory Management** – The OS manages memory for the command structure, buffers and the process image.
- **File Management** – The OS maintains the file descriptor table and maps descriptors to files on disk.
- **Device Management** – The OS switches the stream between the terminal device and the storage device.

- **Security** – The OS enforces file permissions, rejecting a write to a read-only file with EACCES.

Observation

The program successfully parses the input redirection operator, opens the named file, redirects standard input to it, and executes commands such as `wc` and `sort` so that they read their data from the file rather than the keyboard. A deliberately missing file is detected at the `open()` call and reported as "No such file or directory" without terminating the shell, and the saved descriptors restore the terminal afterwards.

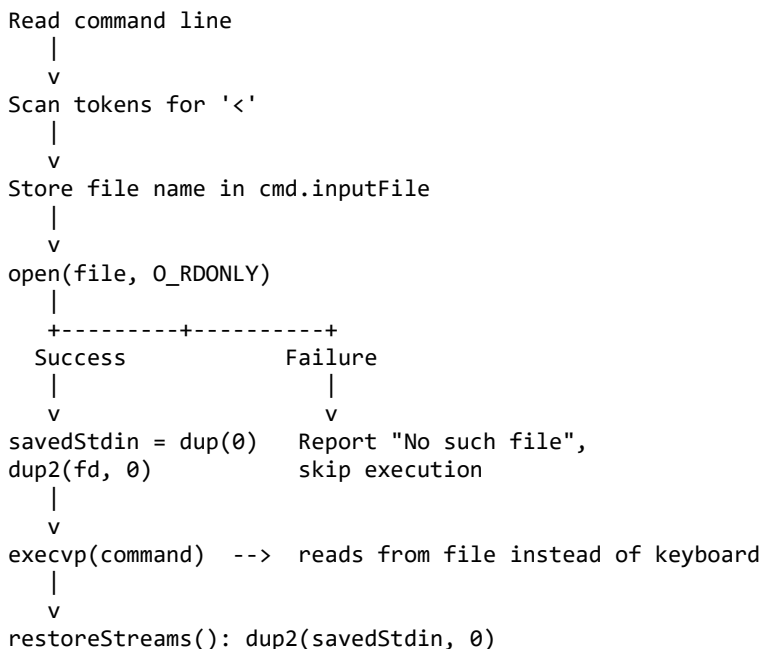
The output redirection module successfully creates the output file with mode 0644, redirects standard output into it using `dup2()`, and writes the command's data to disk instead of the screen. Append mode preserves earlier contents while truncation replaces them, the written data is verified by reading the file back, and an attempt to write to a read-only file is correctly rejected with "Permission denied" while the protected file remains unchanged.

Result

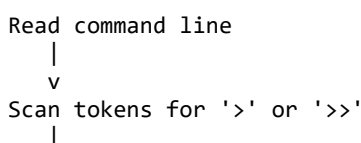
Input redirection was successfully implemented, including parsing the `<` operator, opening files, redirecting standard input, handling missing files, restoring the original streams and testing file-based command input. Output redirection was also successfully implemented, including parsing the `>` and `>>` operators, creating output files, redirecting standard output, handling permission errors, verifying the output data on disk and testing truncating, appending and invalid redirection cases.

Design / Flow

Input Redirection Flow



Output Redirection Flow



```

v
Store file name in cmd.outputFile, set appendMode
|
v
open(file, O_WRONLY | O_CREAT | O_TRUNC/O_APPEND, 0644)
|
+-----+-----+
Success                Failure (EACCES)
|                      |
v                      v
savedStdout = dup(1)    Report "Permission denied",
dup2(fd, 1)            leave file untouched
|
v
execvp(command) --> writes into file instead of terminal
|
v
restoreStreams(): dup2(savedStdout, 1)
|
v
verifyFileContents(): read the file back and display it

```

CODE-

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/wait.h>

#define MAX_CMD_LEN 200
#define MAX_ARGS 20

/* ===== Command structure with redirection info ===== */
typedef struct {
    char *args[MAX_ARGS];
    int argCount;
    char *inputFile; /* file after '<' , NULL if none */
    char *outputFile; /* file after '>' or '>>', NULL if none */
    int appendMode; /* 1 if '>>' was used */
} Command;

/* Saved copies of the original standard streams */
int savedStdin = -1;
int savedStdout = -1;

/* ===== Part 1: Parsing redirection ===== */
int parseRedirection(char *input, Command *cmd)
{
    cmd->argCount = 0;
    cmd->inputFile = NULL;
    cmd->outputFile = NULL;
    cmd->appendMode = 0;

    char *token = strtok(input, " \t");
    while (token != NULL && cmd->argCount < MAX_ARGS - 1)
    {
        if (strcmp(token, "<") == 0)
        {
            token = strtok(NULL, " \t");
            if (token == NULL) { printf("Syntax error: no file after '<'\n"); return 0; }

```

```

        cmd->inputFile = token;
    }
    else if (strcmp(token, ">") == 0 || strcmp(token, ">>") == 0)
    {
        int append = (strcmp(token, ">>") == 0);
        token = strtok(NULL, " \t");
        if (token == NULL) { printf("Syntax error: no file after '>'\n"); return 0; }
        cmd->outputFile = token;
        cmd->appendMode = append;
    }
    else
    {
        cmd->args[cmd->argCount++] = token;
    }
    token = strtok(NULL, " \t");
}
cmd->args[cmd->argCount] = NULL;

if (cmd->argCount == 0) { printf("Syntax error: no command given\n"); return 0; }
return 1;
}

```

```

void printRedirectionLayout(const Command *cmd)

```

```

{
    printf(" Command :");
    for (int i = 0; i < cmd->argCount; i++)
        printf(" %s", cmd->args[i]);
    printf("\n");
    printf(" Input   : %s\n", cmd->inputFile ? cmd->inputFile : "(stdin)");
    printf(" Output  : %s%s\n", cmd->outputFile ? cmd->outputFile : "(stdout)",
        cmd->outputFile && cmd->appendMode ? " [append]" : "");
}

```

```

/* ===== Part 2: Opening files and redirecting ===== */

```

```

/* Opens the input file and redirects stdin to it. Returns 1 on success. */

```

```

int redirectInput(const char *fileName)
{
    int fd = open(fileName, O_RDONLY);
    if (fd < 0)
    {
        printf(" Error: cannot open input file '%s' (%s)\n", fileName, strerror(errno));
        return 0; /* missing file handled gracefully */
    }
    savedStdin = dup(STDIN_FILENO); /* save original stdin */
    dup2(fd, STDIN_FILENO); /* stdin now reads from the file */
    close(fd);
    printf(" stdin redirected from '%s'\n", fileName);
    return 1;
}

```

```

/* Creates/opens the output file and redirects stdout to it. Returns 1 on success. */

```

```

int redirectOutput(const char *fileName, int append)
{
    int flags = O_WRONLY | O_CREAT | (append ? O_APPEND : O_TRUNC);
    int fd = open(fileName, flags, 0644);
    if (fd < 0)
    {
        printf(" Error: cannot open output file '%s' (%s)\n", fileName, strerror(errno));
        return 0; /* permission / path errors handled gracefully */
    }
    savedStdout = dup(STDOUT_FILENO); /* save original stdout */
    dup2(fd, STDOUT_FILENO); /* stdout now writes to the file */
    close(fd);
    return 1;
}

```

```

/* Restores the original standard streams after redirection */
void restoreStreams(void)
{
    if (savedStdin != -1)
    {
        dup2(savedStdin, STDIN_FILENO);
        close(savedStdin);
        savedStdin = -1;
    }
    if (savedStdout != -1)
    {
        dup2(savedStdout, STDOUT_FILENO);
        close(savedStdout);
        savedStdout = -1;
    }
}

/* ===== Executing a command with redirection ===== */
void executeCommand(Command *cmd)
{
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return; }

    if (pid == 0)
    {
        if (cmd->inputFile && !redirectInput(cmd->inputFile))
            _exit(1);
        if (cmd->outputFile && !redirectOutput(cmd->outputFile, cmd->appendMode))
            _exit(1);

        execvp(cmd->args[0], cmd->args);
        restoreStreams();
        fprintf(stderr, "%s: command not found\n", cmd->args[0]);
        _exit(127);
    }
    waitpid(pid, NULL, 0);
}

/* Displays the contents of a file so redirected output can be verified */
void verifyFileContents(const char *fileName)
{
    FILE *fp = fopen(fileName, "r");
    if (fp == NULL)
    {
        printf(" Cannot verify '%s' (%s)\n", fileName, strerror(errno));
        return;
    }
    char line[MAX_CMD_LEN];
    printf(" --- contents of %s ---\n", fileName);
    while (fgets(line, sizeof(line), fp) != NULL)
        printf(" %s", line);
    fclose(fp);
}

/* Creates a small data file used by the input-redirection tests */
void createSampleInputFile(const char *fileName)
{
    FILE *fp = fopen(fileName, "w");
    if (fp == NULL) { perror("fopen"); return; }
    fprintf(fp, "delta\napple\ncherry\nbanana\napple\n");
    fclose(fp);
    printf("Sample input file '%s' created with 5 lines.\n", fileName);
}

/* ===== Part 3: Test drivers ===== */
void testInputRedirection(void)
{

```

```

printf("\n=== Part 1: Testing Input Redirection ===\n");
createSampleInputFile("input.txt");

char *tests[] = {
    "wc -l < input.txt",
    "sort < input.txt",
    "cat < missing.txt" /* deliberately missing file */
};
int n = sizeof(tests) / sizeof(tests[0]);

for (int i = 0; i < n; i++)
{
    char buffer[MAX_CMD_LEN];
    strncpy(buffer, tests[i], MAX_CMD_LEN - 1);
    buffer[MAX_CMD_LEN - 1] = '\0';

    printf("\nInput: \"%s\"\n", tests[i]);
    Command cmd;
    if (!parseRedirection(buffer, &cmd))
        continue;
    printRedirectionLayout(&cmd);
    printf(" Output:\n");
    executeCommand(&cmd);
}
restoreStreams();
printf("\nStandard streams restored after input tests.\n");
}

void testOutputRedirection(void)
{
    printf("\n=== Part 2: Testing Output Redirection ===\n");

    /* Create a read-only directory-less file to trigger a permission error */
    FILE *fp = fopen("readonly.txt", "w");
    if (fp) { fprintf(fp, "protected\n"); fclose(fp); }
    chmod("readonly.txt", 0444);
    printf("File 'readonly.txt' created with permissions 0444 (read-only).\n");

    char *tests[] = {
        "echo Hello Operating Systems > output.txt",
        "echo Second line appended >> output.txt",
        "echo blocked > readonly.txt" /* deliberately permission denied */
    };
    int n = sizeof(tests) / sizeof(tests[0]);

    for (int i = 0; i < n; i++)
    {
        char buffer[MAX_CMD_LEN];
        strncpy(buffer, tests[i], MAX_CMD_LEN - 1);
        buffer[MAX_CMD_LEN - 1] = '\0';

        printf("\nInput: \"%s\"\n", tests[i]);
        Command cmd;
        if (!parseRedirection(buffer, &cmd))
            continue;
        printRedirectionLayout(&cmd);
        executeCommand(&cmd);
        verifyFileContents(cmd.outputFile);
    }
    restoreStreams();
    printf("\nStandard streams restored after output tests.\n");
    chmod("readonly.txt", 0644);
}

/* ===== Main ===== */
int main(void)

```

```
{
    setvbuf(stdout, NULL, _IONBF, 0); /* keep child and parent output in order */
    testInputRedirection();
    testOutputRedirection();
    return 0;
}
```

Compilation and Execution

1. Compile

```
gcc question13.c -o question13
```

2. Execute

```
./question13
```

Sample Output

```
teja@Hasini:~$ nano redirection.c
teja@Hasini:~$ gcc redirection.c -o redirection
teja@Hasini:~$ ./redirection
```

=== Part 1: Testing Input Redirection ===

Sample input file 'input.txt' created with 5 lines.

Input: "wc -l < input.txt"

Command : wc -l

Input : input.txt

Output : (stdout)

Output:

stdin redirected from 'input.txt'

5

Input: "sort < input.txt"

Command : sort

Input : input.txt

Output : (stdout)

Output:

stdin redirected from 'input.txt'

apple

apple

banana

cherry

delta

Input: "cat < missing.txt"

Command : cat

Input : missing.txt

Output : (stdout)

Output:

Error: cannot open input file 'missing.txt' (No such file or directory)

Standard streams restored after input tests.

```
=== Part 2: Testing Output Redirection ===  
File 'readonly.txt' created with permissions 0444 (read-only).
```

```
Input: "echo Hello Operating Systems > output.txt"
```

```
Command : echo Hello Operating Systems
```

```
Input    : (stdin)
```

```
Output   : output.txt
```

```
--- contents of output.txt ---
```

```
Hello Operating Systems
```

```
Input: "echo Second line appended >> output.txt"
```

```
Command : echo Second line appended
```

```
Input    : (stdin)
```

```
Output   : output.txt [append]
```

```
--- contents of output.txt ---
```

```
Hello Operating Systems
```

```
Second line appended
```

```
Input: "echo blocked > readonly.txt"
```

```
Command : echo blocked
```

```
Input    : (stdin)
```

```
Output   : readonly.txt
```

```
Error: cannot open output file 'readonly.txt' (Permission denied)
```

```
--- contents of readonly.txt ---
```

```
protected
```

```
Standard streams restored after output tests.
```

```
teja@Hasini:~$ cat input.txt
```

```
delta
```

```
apple
```

```
cherry
```

```
banana
```

```
apple
```

```

teja@Hasini:~$ cat input.txt
delta
apple
cherry
banana
apple
teja@Hasini:~$ cat output.txt
Hello Operating Systems
Second line appended
teja@Hasini:~$ ls -l
total 268
drwxr-xr-x 2 teja teja 4096 Aug  1 10:02 Experiment-2
-rw----- 1 teja teja 1034 Aug  8 10:06 filecopy.c.save
-rw----- 1 teja teja 1180 Aug  8 10:07 filecopy.c.save.1
-rwxr-xr-x 1 teja teja 16080 Aug 18 06:04 fork
-rw-r--r-- 1 teja teja  471 Aug 18 06:04 fork.c
-rwxr-xr-x 1 teja teja 16256 Aug  8 10:08 hello
-rw-r--r-- 1 teja teja 1180 Aug  8 10:08 hello.c
-rw-r--r-- 1 teja teja   32 Sep 17 03:42 input.txt
drwxr-xr-x 3 teja teja 4096 Jul 30 05:04 linux
-rwxr-xr-x 1 teja teja 16472 Jul 30 04:56 os_task
-rw-r--r-- 1 teja teja  896 Jul 30 04:56 os_task.c
-rw-r--r-- 1 teja teja   45 Sep 17 03:42 output.txt
-rwxr-xr-x 1 teja teja 16336 Aug  8 10:00 parser
-rw-r--r-- 1 teja teja  3258 Aug  8 10:00 parser.c
-rwxr-xr-x 1 teja teja 16680 Aug 27 05:44 path_resolver
-rw-r--r-- 1 teja teja  2595 Aug 27 05:44 path_resolver.c
-rwxr-xr-x 1 teja teja 17312 Sep 17 03:31 question
-rw-r--r-- 1 teja teja  7352 Sep 17 03:31 question.c
-rwxr-xr-x 1 teja teja 16792 Sep 17 03:13 question7
-rw-r--r-- 1 teja teja  2181 Sep 17 03:13 question7.c
-rwxr-xr-x 1 teja teja 17264 Sep 17 03:21 question9
-rw-r--r-- 1 teja teja  4530 Sep 17 03:21 question9.c
-rwxr-xr-x 1 teja teja 16304 Aug 25 03:49 quotes
-rw-r--r-- 1 teja teja  3576 Aug 25 03:49 quotes.c

```

Conclusion

This experiment demonstrates input and output redirection in a custom shell, including parsing of the '<', '>' and '>>' operators, opening files with `open()`, replacing the standard streams with `dup2()`, and restoring them from descriptors saved with `dup()`. It also demonstrates robust error handling for missing input files and for output files that cannot be written because of permissions, together with verification of the data actually stored on disk. Together, these illustrate important Operating System concepts such as file descriptor management, the role of the descriptor table across `fork()` and `execvp()`, device independence of the standard streams, and permission enforcement by the kernel.